

Design Doc Template

Your Name

December 17, 2025

Contents

1	Template Preamble	2
2	Context and Scope	2
2.1	Goals	2
2.2	Non-Goals	3
2.3	Value Added	3
2.4	Stakeholders	3
2.5	Dependencies	3
3	Design	3
3.1	Overview and System Context Diagram	3
3.2	Implementation Design	3
3.3	Risks and Problems	4
4	Alternative Considered	4
5	Cross-Cutting Concerns	4
6	Completion Criteria	4
6.1	Definition of Done	4
6.2	Performance Criteria	5
6.3	Implementation Plan	5
7	Future Work	5

1 Template Preamble

Design docs are written to solve problems. This design doc was inspired from Google¹ and has been modified over the years for team leaders and engineering practitioners. Writing design docs is overhead. The decision whether to write a design doc comes down to the core trade-off of deciding whether the benefits in organizational consensus around design, documentation, senior review, etc. outweigh the extra work of creating the doc. At the center of that decision lies whether the solution to the design problem is ambiguous—because of problem complexity or solution complexity, or both.

Design docs should be sufficiently detailed but short enough to actually be read by busy people. The sweet spot for a larger project seems to be around 10-20ish pages. If you get way beyond that, it might make sense to split up the problem into more manageable sub problems. It should also be noted that it is absolutely possible to write a 1-3 page “mini design doc”. This is especially helpful for incremental improvements or sub tasks in an agile project—you still do all the same steps as for a longer doc, just keep things more terse and focused on a limited problem set.

The design doc is *the place to write down the trade-offs* you made in designing your solution. Focus on those trade-offs to produce a useful document with long-term value. That is, given the context (facts), goals and non-goals (requirements), the design doc is the place to suggest solutions and show why a particular solution best satisfies those goals.

The point of writing a document over a more formal medium is to provide the flexibility to express the problem set at hand in an appropriate manner. Because of this, there is no explicit guidance for how to actually describe the design. Having said that, a few best practices and repeating topics have emerged that make sense for a large percentage of design docs.

¹<https://www.industrialempathy.com/posts/design-docs-at-google/>

2 Context and Scope

This section gives the reader a very rough overview of the landscape in which the new system is being built and what is actually being built. This isn't a requirements doc. Keep it succinct! The goal is that readers are brought up to speed but some previous knowledge can be assumed and detailed info can be linked to. This section should be entirely focused on objective background facts.

2.1 Goals

A short list of bullet points of what the goals of the system are, and, sometimes more importantly, what non-goals are.

- Item A
- Item B

2.2 Non-Goals

Note, that non-goals aren't negated goals like "The system shouldn't crash", but rather things that could reasonably be goals, but are explicitly chosen not to be goals. A good example would be "ACID compliance"; when designing a database, you'd certainly want to know whether that is a goal or non-goal. And if it is a non-goal you might still select a solution that provides it, if it doesn't introduce trade-offs that prevent achieving the goals.

- Item C
- Item D

2.3 Value Added

What value does this add to the overall system and what benefits can be derived once this is implemented?

2.4 Stakeholders

List all the stakeholders who will be impacted and must be notified prior to implementation:

- Stakeholder A
- Stakeholder B

2.5 Dependencies

Identify what pieces need to be in place before for this design can be implemented.

3 Design

3.1 Overview and System Context Diagram

In many docs a system-context diagram can be very useful. Such a diagram shows the system as part of the larger technical landscape and allows readers to contextualize the new design given its environment that they are already familiar with.

3.2 Implementation Design

Pseudo-code, data storage, APIs, ICDs, etc.

3.3 Risks and Problems

What are some risks in implementation, schedule, resourcing and cost that can arise?

4 Alternative Considered

This section lists alternative designs that would have reasonably achieved similar outcomes. The focus should be on the trade-offs that each respective design makes and how those trade-offs led to the decision to select the design that is the primary topic of the document.

While it is fine to be succinct about solution that ended up not being selected, this section is one of the most important ones as it shows very explicitly why the selected solution is the best given the project goals and how other solutions, that the reader may be wondering about, introduce trade-offs that are less desirable given the goals.

5 Cross-Cutting Concerns

This is where your organization can ensure that certain cross-cutting concerns such as security, privacy, and observability are always taken into consideration. These are often relatively short sections that explain how the design impacts the concern and how the concern is addressed. Teams should standardize what these concerns are in their case.

Due to their importance, Google projects are required to have a dedicated privacy design doc, and there are dedicated reviews for privacy and security. While the reviews are only required to be completed by the time a project launches, it is best practice to engage with privacy and security teams as early as possible to ensure that designs take them into account from the ground up. In case of dedicated docs for these topics, the central design doc can, of course, reference them instead of going into detail.

6 Completion Criteria

6.1 Definition of Done

What pieces need to be completed in order for the work to be deemed done.

- Design doc reviewed and approved by stakeholders
- Testing (80% ccoverage)
- Deployed

6.2 Performance Criteria

How well does the solution need to perform in order to be deemed sufficient.

- Working at a rate of 2Hz
- Consuming 2GB or memory

6.3 Implementation Plan

Sequence of tasks that can be turned into work tickets to implement the solution.

Index	Size	Description	Completed
1	S (2 weeks)	Enter a description of the work package item.	No
2	M (4-6 weeks)	Enter a description of the work package item.	No
3	L (6+ weeks)	Enter a description of the work package item.	No

7 Future Work

What work needs to be done in the future to improve the solution and/or expand the system?